



Más SQL

Una vez que hemos revisado las instrucciones básicas para comenzar a trabajar con nuestra base de datos (como definir cómo se almacenarán los datos, crear nuevos registros, recuperar información, actualizar registros y eliminar algunos de ellos) avancemos ahora a cómo realizar consultas un poco más elaboradas. Por ejemplo, veremos **cómo utilizar el resultado de una consulta (*query*) como parámetro para otra**, exploraremos formas de simplificar el trabajo con consultas complejas una vez que han sido escritas, y también revisaremos cómo **modificar el esquema** de nuestra base de datos.

Anteriormente mencionamos que, en las bases de datos relacionales, se espera que la estructura de los datos no cambie con el tiempo. Sin embargo, si las condiciones del mundo real cambian, la base de datos debe evolucionar para adaptarse a esos cambios.

Para continuar trabajando con consultas, retomemos el concepto del valor `NULL`, mencionado en el tema anterior. Utilizamos `NULL` para denotar la ausencia de un valor, pero en el contexto de bases de datos, suele tener tres posibles interpretaciones:

- 1 **Valor desconocido:** el valor existe, pero no se conoce, o no se sabe si existe o no.
- 2 **Valor no disponible:** el valor existe, pero intencionalmente no se registra.
- 3 **Valor no aplicable:** el valor no aplica para una tupla específica o está indefinido para esa tupla.

Por lo general, no es posible determinar cuál de estos tres significados se le quiso dar al valor `NULL`. Por ejemplo, si el número de teléfono de una persona es `NULL`, esto podría significar:

- a **Valor desconocido:** no sabemos si el número existe o simplemente no lo conocemos.
- b **Valor no disponible:** el número existe, pero no se desea almacenar en la base de datos, posiblemente por razones de privacidad.
- c **Valor no aplicable:** estamos registrando estudiantes de un nivel educativo muy temprano, donde los niños no utilizan un número telefónico.

Pasemos ahora a la parte práctica. El valor `NULL` se trata de manera especial. Por ejemplo, para seleccionar las tuplas donde un atributo sea `NULL`, no utilizamos la sintaxis `atributo = NULL`, sino `atributo IS NULL`.

Recordando el ejemplo que hemos trabajado de la agencia de autos, observemos cómo están las tablas de `marca`, `submarca` y `auto`:

```
mysql> SELECT * FROM marca;
+-----+-----+
| id_marca | marca      |
+-----+-----+
|          1 | VOLKSWAGEN |
|          2 | HONDA      |
|          3 | TOYOTA     |
|          4 | FORD       |
|          5 | MAZDA      |
+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM submarca;
+-----+-----+
| submarca | id_marca |
+-----+-----+
| VITARA   | NULL     |
| WRX      | NULL     |
| POLO     | 1        |
| TIGUAN   | 1        |
| CIVIC    | 2        |
| COROLLA  | 3        |
+-----+-----+
6 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM auto;
```

vin	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	2020	BLANCO	MANUAL	50000

```
6 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM marca;
```

id_marca	marca
1	VOLKSWAGEN
2	HONDA
3	TOYOTA
4	FORD
5	MAZDA

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM submarca;
```

submarca	id_marca
VITARA	NULL
WRX	NULL
POLO	1
TIGUAN	1
CIVIC	2
COROLLA	3

```
6 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM auto;
```

vin	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	2020	BLANCO	MANUAL	50000

```
6 rows in set (0.00 sec)
```


Podemos ver que en la tabla `submarca` hay algunas tuplas donde el campo `id_marca` es `NULL`. Si queremos consultar estas submarcas, la consulta sería así:

```
mysql> SELECT * FROM submarca WHERE id_marca IS NULL;
+-----+-----+
| submarca | id_marca |
+-----+-----+
| VITARA   | NULL     |
| WRX      | NULL     |
+-----+-----+
2 rows in set (0.00 sec)
```

Y si queremos ver aquellas donde `id_marca` no sea `NULL`, la consulta quedaría de este modo:

```
mysql> SELECT * FROM submarca WHERE id_marca IS NOT NULL;
+-----+-----+
| submarca | id_marca |
+-----+-----+
| POLO     | 1        |
| TIGUAN   | 1        |
| CIVIC    | 2        |
| COROLLA  | 3        |
+-----+-----+
4 rows in set (0.00 sec)
```

Debido a que el valor `NULL` se trata de forma especial, cualquier operación de comparación en la que intervenga un `NULL` produce un resultado **desconocido**. Por eso, SQL utiliza una lógica de **tres valores**: `TRUE`, `FALSE` y `UNKNOWN`, en lugar de la lógica booleana tradicional que solo contempla `TRUE` y `FALSE`.

Por lo tanto, definamos a continuación los resultados o las tablas de valor que aplican cuando usamos los conectores lógicos AND, OR y NOT:

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

NOT	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Veamos un ejemplo de cómo funciona esto. Recordando la tabla de autos:

```
mysql> SELECT * FROM auto;
```

vin	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	42000
3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	2020	BLANCO	MANUAL	50000

```
6 rows in set (0.00 sec)
```

Agreguemos un par de autos más, pero sin un valor para `recorrido`:

```
INSERT INTO auto VALUES
("3VWJ7DY65TD3", "POLO", 2020, "ROJO FLASH", "AUTOMATI-
CA", NULL),
("6TDK4K57KFD5", "COROLLA", 2018, "PLATA", "MANUAL",
NULL);
```

```
mysql> INSERT INTO auto VALUES
-> ("3VWJ7DY65TD3", "POLO", 2020, "ROJO FLASH", "AUTOMATICA", NULL),
-> ("6TDK4K57KFD5", "COROLLA", 2018, "PLATA", "MANUAL", NULL);
Query OK, 2 rows affected (0.17 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM auto;
```

vin	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	42000
3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	2020	BLANCO	MANUAL	50000
3VWJ7DY65TD3	POLO	2020	ROJO FLASH	AUTOMATICA	NULL
6TDK4K57KFD5	COROLLA	2018	PLATA	MANUAL	NULL

8 rows in set (0.00 sec)

Si hacemos una consulta para ver los autos que tienen menos de 100,000 kilómetros, el código es:

```
select * from auto WHERE recorrido < 100000;
```

```
mysql> select * from auto WHERE recorrido < 100000;
```

vin	submarca	modelo	color	transmision	recorrido
HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	42000
6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000
9DTF6S5SG5G5	WRX	2020	BLANCO	MANUAL	50000

3 rows in set (0.00 sec)

Y si consultamos los que tienen más de 100,000 kilómetros, el resultado es:

```
select * from auto WHERE recorrido > 100000;
```

```
mysql> select * from auto WHERE recorrido > 100000;
```

vin	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000
JD76F6SDF7DF	VITARA	2016	BLANCO	AUTOMATICA	213000

```
3 rows in set (0.00 sec)
```

Observa que los autos recién insertados no aparecen en ninguna de las dos consultas. Esto se debe a que el valor recorrido es NULL, y como el resultado de la comparación es UNKNOWN, no se cumple ninguna de las condiciones.

Ahora bien, ¿qué pasa cuando usamos **operadores lógicos**? Como vimos antes, TRUE OR UNKNOWN resulta en TRUE. Entonces, si consultamos autos cuyo modelo sea mayor a 2015 o cuyo recorrido sea menor a 100,000 kilómetros:

```
SELECT * FROM auto WHERE modelo > 2015 OR recorrido < 100000;
```

```
mysql> SELECT * FROM auto WHERE modelo > 2015 OR recorrido < 100000;
```

vin	submarca	modelo	color	transmision	recorrido
HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	42000
6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	2020	BLANCO	MANUAL	50000
3VWJ7DY65TD3	POLO	2020	ROJO FLASH	AUTOMATICA	NULL
6TDK4K57KFD5	COROLLA	2018	PLATA	MANUAL	NULL

```
6 rows in set (0.00 sec)
```

En este caso, tanto el COROLLA PLATA como el POLO ROJO aparecen en el resultado debido a que `TRUE(modelo > 2015) OR UNKNOWN(recorrido < 100000)` es `TRUE`.

| Uso de consultas anidadas (*subqueries*)

Con frecuencia escribimos consultas que se vuelven muy largas o complejas porque necesitamos hacer una consulta, y con la información obtenida, debemos realizar otra.

Para esto, podemos utilizar **consultas anidadas**. La propiedad de cerradura del álgebra relacional establece que el resultado de una operación sobre tablas también es una tabla, lo cual permite que una consulta esté dentro de otra.

Veamos un ejemplo siguiendo con el contexto de la agencia que hemos estado trabajando. Supongamos que tenemos esta consulta que muestra los autos disponibles en nuestra agencia:

```
SELECT
    vin, auto.submarca, marca, modelo, color, transmision, recorrido
FROM
    auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
    JOIN marca ON submarca.id_marca = marca.id_marca;
```

```
mysql> SELECT
->   vin, auto.submarca, marca, modelo, color, transmission, recorrido
-> FROM
->   auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca;
```

vin	submarca	marca	modelo	color	transmission
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL

4 rows in set (0.00 sec)

Ahora, supongamos que, de esta consulta, nos interesa conocer cuáles son los autos que tienen menos de 100,000 kilómetros. Esto podemos hacerlo **anidando consultas**: le asignamos un nombre a la consulta que ya teníamos, y al resultado de esa operación le pedimos que nos muestre los autos con menos de 100,000 kilómetros.

Veamos cómo se hace:

```
mysql> SELECT * FROM (
->   SELECT
->     vin, auto.submarca, marca, modelo, color, transmission, recorrido
->   FROM
->     auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
->   JOIN marca ON submarca.id_marca = marca.id_marca
-> ) AS autos_disponibles
-> WHERE recorrido < 100000 ;
```

vin	submarca	marca	modelo	color	transmission	recorrido
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000

2 rows in set (0.00 sec)

En este caso, iniciamos con `SELECT * FROM`, como cualquier otra consulta, pero en lugar de poner una tabla existente, escribimos nuestra primera consulta entre paréntesis. Luego, escribimos `AS <nombre_de_la_subconsulta>`, en nuestro caso, `autos_disponibles`. Cabe destacar que este nombre es temporal, y que **NO** se crea ninguna estructura ni se puede referenciar posteriormente el resultado de esa consulta. Más adelante veremos cómo guardar resultados de consultas complejas usando vistas.

De esta manera, podemos anidar no solo una consulta dentro de otra, si no que podríamos construir, por ejemplo, una consulta que utilice el resultado de la consulta previamente mostrada.

Cuando el resultado de una consulta es un solo valor (un **valor escalar**), podemos usarlo para hacer comparaciones. Por ejemplo, en nuestra consulta de los autos disponibles, si queremos ver los autos que son del mismo año o más nuevos que el VOLKSWAGEN POLO, podríamos hacer una consulta para obtener el modelo del VOLKSWAGEN POLO, y eso utilizarlo como **parámetro** para la consulta que nos interesa.

Esto quedaría como:

```
SELECT
    vin, auto.submarca, marca, modelo, color, transmision, recorrido
FROM
    auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
    JOIN marca ON submarca.id_marca = marca.id_marca
WHERE
    modelo >= (SELECT modelo FROM auto RIGHT JOIN submarca
ON auto.submarca = submarca.submarca JOIN marca ON submarca.
id_marca = marca.id_marca WHERE marca.marca="VOLKSWAGEN" and
submarca.submarca= "POLO")
;
```


Es cierto que podríamos haber escrito directamente `modelo >= 2015`, pero estamos ilustrando el uso de este tipo de consultas anidadas.

Cuando el resultado de una consulta es un **conjunto de valores**, podemos usar `IN` en la consulta externa para filtrar aquellos registros cuyo atributo esté dentro del conjunto devuelto.

Por ejemplo, los números de serie (VIN) de los vehículos VOLKSWAGEN inician con "3VW". Podemos hacer una consulta para visualizar los VIN de los vehículos que inicien con "3VW", y podemos hacer una consulta que nos muestre toda la información de los autos cuyo número de serie inicia con "3VW", como se muestra a continuación:

- 1 Primero, consultamos los autos que inician con "3VW". Esto lo podemos hacer con la condición `WHERE vin LIKE "3VW%"`.
- 2 Cuando tenemos consultas que involucran atributos de tipo cadena, como `CHAR()` o `VARCHAR()`, podemos especificar que queremos que:

- **El valor inicie con "3VW"**. Usamos `WHERE vin LIKE "3VW%"`

- **El valor termine con "3VW"**. Usamos `WHERE vin LIKE "%3VW"`

- **El valor contenga en alguna parte "3VW"**. Usamos `WHERE vin LIKE "%3VW%"`

```
SELECT vin FROM auto WHERE vin LIKE "3VW%";
```

```
mysql> SELECT vin FROM auto WHERE vin LIKE "3VW%";
```

```
+-----+
| vin      |
+-----+
| 3VWU67FR7RE3 |
| 3VWGCISDS7D7 |
+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT vin FROM auto WHERE vin LIKE "3VW%";
+-----+
| vin      |
+-----+
| 3VWU67FR7RE3 |
| 3VWGCISDS7D7 |
+-----+
2 rows in set (0.00 sec)
```

Ahora, anidando la consulta, tenemos:

```
SELECT
    vin, auto.submarca, marca, modelo, color, transmision, recorrido
FROM
    auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
    JOIN marca ON submarca.id_marca = marca.id_marca
WHERE
    vin IN (SELECT vin FROM auto WHERE vin LIKE "3VW%")
;
```

```
mysql> SELECT
-> vin, auto.submarca, marca, modelo, color, transmision, recorrido
-> FROM
-> auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca
-> WHERE
-> vin IN (SELECT vin FROM auto WHERE vin LIKE "3VW%")
-> ;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000

2 rows in set (0.00 sec)

```
mysql> SELECT
->   vin, auto.submarca, marca, modelo, color, transmision, recorrido
-> FROM
->   auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca
-> WHERE
->   vin IN (SELECT vin FROM auto WHERE vin LIKE "3VW%")
-> ;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000

2 rows in set (0.01 sec)

También podemos utilizar `IN` para especificar un conjunto de valores con los que queremos comparar un atributo:

```
mysql> SELECT
->   vin, auto.submarca, marca, modelo, color, transmision, recorrido
-> FROM
->   auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca
-> WHERE
->   vin IN ("3VWU67FR7RE3", "3VWGCISDS7D7")
-> ;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000

2 rows in set (0.00 sec)

```
mysql> SELECT
->   vin, auto.submarca, marca, modelo, color, transmission, recorrido
-> FROM
->   auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca
-> WHERE
->   vin IN ("3VWU67FR7RE3", "3VWGCISDS7D7")
-> ;
```

vin	submarca	marca	modelo	color	transmission	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000

2 rows in set (0.00 sec)

```
SELECT
  vin, auto.submarca, marca, modelo, color, transmission, recorrido
FROM
  auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
  JOIN marca ON submarca.id_marca = marca.id_marca
WHERE
  recorrido IN (30000, 40000);
```

```
mysql> SELECT
->   vin, auto.submarca, marca, modelo, color, transmission, recorrido
-> FROM
->   auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca
-> WHERE
->   recorrido IN (30000, 40000)
-> ;
```

vin	submarca	marca	modelo	color	transmission	recorrido
H6G6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000

2 rows in set (0.00 sec)

Cuando utilizamos `IN`, los resultados deben coincidir con alguno de los valores especificados en la lista que está después del `IN`:

```
SELECT
  vin, auto.submarca, marca, modelo, color, transmision, recorrido
FROM
  auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
  JOIN marca ON submarca.id_marca = marca.id_marca
WHERE
  recorrido BETWEEN 30000 AND 40000
;
```

```
mysql> SELECT
->   vin, auto.submarca, marca, modelo, color, transmision, recorrido
-> FROM
->   auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
-> JOIN marca ON submarca.id_marca = marca.id_marca
-> WHERE
-> recorrido BETWEEN 30000 AND 40000
-> ;
```

vin	submarca	marca	modelo	color	transmision	recorrido
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000

2 rows in set (0.01 sec)

Por otro lado, cuando utilizamos `BETWEEN`, los valores en el resultado son los que **son mayores o iguales al primer parámetro y menores o iguales al segundo**.

Creación y uso de vistas (VIEWS)

En los ejemplos que hemos revisado hasta ahora, hemos utilizado consultas que han ido creciendo en longitud y complejidad. Hemos involucrado dos o tres `JOIN` entre varias tablas, y es posible que, a medida que el escenario del mundo real que queremos modelar se vuelva más complejo, las consultas también aumenten en dificultad. Es en este punto donde las **vistas** pueden ayudarnos.

Una **vista** en SQL es una **tabla virtual** construida a partir del resultado de una consulta SQL.

La sintaxis y los detalles de la instrucción `CREATE VIEW` pueden consultarse en la documentación oficial según el sistema de gestión de bases de datos (DBMS) correspondiente. Para este caso, en la sección de **Material adicional** podrás encontrar un enlace para acceder a la documentación oficial de MySQL.

La sintaxis más común para crear una vista es la siguiente:

```
CREATE VIEW nombre_de_la_vista AS consulta_sql
```

En el escenario de la agencia de autos que hemos venido trabajando, contamos con tablas para marcas, submarcas y autos. Una consulta común podría ser: “Muéstrame todas las marcas y submarcas registradas”.

Para obtener esta información, necesitamos realizar un `JOIN` entre las tablas de marcas y submarcas. Recordemos que **en MySQL no se permite el uso del operador `OUTER JOIN`**, por lo que lograremos el mismo resultado mediante la combinación (`UNION`) de un `LEFT JOIN` y un `RIGHT JOIN`:

Veamos cómo quedaría la consulta:

```
SELECT
    marca, submarca
FROM marca LEFT JOIN submarca ON marca.id_marca = submarca.id_marca

UNION

SELECT
    marca, submarca
FROM marca RIGHT JOIN submarca ON marca.id_marca = submarca.id_marca

;
```

```
mysql> SELECT
->     marca, submarca
->     FROM marca LEFT JOIN submarca ON marca.id_marca = submarca.id_marca
->
->     UNION
->
->     SELECT
->     marca, submarca
->     FROM marca RIGHT JOIN submarca ON marca.id_marca = submarca.id_marca
->
->     ;
```

```
+-----+-----+
| marca      | submarca |
+-----+-----+
| VOLKSWAGEN | POLO     |
| VOLKSWAGEN | TIGUAN   |
| HONDA       | CIVIC    |
| TOYOTA      | COROLLA  |
| FORD        | NULL     |
| MAZDA       | NULL     |
| NULL       | VITARA   |
| NULL       | WRX      |
+-----+-----+
8 rows in set (0.00 sec)
```



```
mysql> SELECT
->   marca, submarca
-> FROM marca LEFT JOIN submarca ON marca.id_marca = submarca.id_marca
->
-> UNION
->
-> SELECT
->   marca, submarca
-> FROM marca RIGHT JOIN submarca ON marca.id_marca = submarca.id_marca
->
-> ;
```

marca	submarca
VOLKSWAGEN	POLO
VOLKSWAGEN	TIGUAN
HONDA	CIVIC
TOYOTA	COROLLA
FORD	NULL
MAZDA	NULL
NULL	VITARA
NULL	WRX

8 rows in set (0.00 sec)

Ahora, vamos a guardar el resultado de esta consulta como una vista llamada `todas_marca_submarca`:

```
mysql> CREATE VIEW todas_marca_submarca AS
->
->   SELECT
->     marca, submarca
->   FROM marca LEFT JOIN submarca ON marca.id_marca = submarca.id_marca
->
->   UNION
->
->   SELECT
->     marca, submarca
->   FROM marca RIGHT JOIN submarca ON marca.id_marca = submarca.id_marca
->
-> ;
```

Query OK, 0 rows affected (0.03 sec)

Una vez creada la vista, podemos utilizarla como si fuera otra tabla más dentro de nuestra base de datos.

```
mysql> SELECT * FROM todas_marca_submarca;
```

marca	submarca
VOLKSWAGEN	POLO
VOLKSWAGEN	TIGUAN
HONDA	CIVIC
TOYOTA	COROLLA
FORD	NULL
MAZDA	NULL
NULL	VITARA
NULL	WRX

8 rows in set (0.01 sec)

Veamos otro ejemplo. Anteriormente escribimos una consulta para ver cuáles eran los autos disponibles en un momento dado. Escribamos ahora la instrucción para crear una vista con el resultado de esta consulta:

```
CREATE VIEW autos_disponibles AS
    SELECT
        vin, auto.submarca, marca, modelo, color, transmision, recorrido
    FROM
        auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
        JOIN marca ON submarca.id_marca = marca.id_marca
;
```

```
mysql> CREATE VIEW autos_disponibles AS
-> SELECT
->     vin, auto.submarca, marca, modelo, color, transmision, recorrido
-> FROM
->     auto RIGHT JOIN submarca ON auto.submarca = submarca.submarca
->     JOIN marca ON submarca.id_marca = marca.id_marca
-> ;CREATE VIEW autos_disponibles AS
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM autos_disponibles;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000

4 rows in set (0.01 sec)

Ya con la vista creada, podemos realizar consultas más simples sobre ella. Por ejemplo, para obtener solo los autos con menos de 100,000 kilómetros recorridos:

```
SELECT * FROM autos_disponibles WHERE recorrido < 100000 ;
```

```
mysql> SELECT * FROM autos_disponibles WHERE recorrido < 100000 ;
```

vin	submarca	marca	modelo	color	transmision	recorrido
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000

2 rows in set (0.00 sec)

Veamos otro caso. En el ejemplo anterior, solo se mostraban los autos que tenían una marca asociada. Previamente habíamos escrito una consulta para obtener la información de los autos de la agencia, aunque no tuvieran una marca asociada, de esta manera:

```
SELECT auto.vin AS vin,  
       auto.submarca AS submarca,  
       marca.marca AS marca,  
       auto.modelo AS modelo,
```

```

    auto.color AS color,
    auto.transmission AS transmission,
    auto.recorrido AS recorrido
FROM ((auto LEFT JOIN submarca
      ON((auto.submarca = submarca.submarca))) LEFT
JOIN marca ON((submarca.id_marca = marca.id_marca)));

```

```

mysql> SELECT auto.vin AS vin,
->      auto.submarca AS submarca,
->      marca.marca AS marca,
->      auto.modelo AS modelo,
->      auto.color AS color,
->      auto.transmission AS transmission,
->      auto.recorrido AS recorrido
-> FROM ((auto LEFT JOIN submarca
->      ON((auto.submarca = submarca.submarca))) LEFT JOIN marca ON((submarca.id_marca
marca.id_marca)));
+-----+-----+-----+-----+-----+-----+-----+
| vin          | submarca | marca    | modelo | color        | transmission | recorrido |
+-----+-----+-----+-----+-----+-----+-----+
| 3VWU67FR7RE3 | POLO     | VOLKSWAGEN | 2015   | GRIS PLATINO | MANUAL       | 125000   |
| HDG6JBSD7FDT | CIVIC    | HONDA      | 2020   | PLATA DIAMANTE | AUTOMATICA  | 40000    |
| 3VWGCISDS7D7 | TIGUAN   | VOLKSWAGEN | 2012   | BLANCO       | AUTOMATICA  | 132000   |
| 6TDG675E54GY | COROLLA  | TOYOTA     | 2022   | ROJO         | MANUAL       | 30000    |
| JD76F6SDF7DF | VITARA   | NULL       | 2016   | BLANCO       | AUTOMATICA  | 213000   |
| 9DTF6S5SG5G5 | WRX      | NULL       | 2020   | BLANCO       | MANUAL       | 50000    |
+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

```

mysql> SELECT auto.vin AS vin,
->      auto.submarca AS submarca,
->      marca.marca AS marca,
->      auto.modelo AS modelo,
->      auto.color AS color,
->      auto.transmission AS transmission,
->      auto.recorrido AS recorrido
-> FROM ((auto LEFT JOIN submarca
->      ON((auto.submarca = submarca.submarca))) LEFT JOIN marca
ON((submarca.id_marca = marca.id_marca)));

```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	NULL	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	NULL	2020	BLANCO	MANUAL	50000

6 rows in set (0.00 sec)

Podemos guardar esta consulta como una vista:

```
CREATE VIEW V1 AS
SELECT auto.vin AS vin,
       auto.submarca AS submarca,
       marca.marca AS marca,
       auto.modelo AS modelo,
       auto.color AS color,
       auto.transmision AS transmision,
       auto.recorrido AS recorrido
FROM ((auto LEFT JOIN submarca
      ON((auto.submarca = submarca.submarca))) LEFT
JOIN marca ON((submarca.id_marca = marca.id_marca)))
;
```

```
mysql> CREATE VIEW V1 AS
-> SELECT auto.vin AS vin,
->       auto.submarca AS submarca,
->       marca.marca AS marca,
->       auto.modelo AS modelo,
->       auto.color AS color,
->       auto.transmision AS transmision,
->       auto.recorrido AS recorrido
-> FROM ((auto LEFT JOIN submarca
->       ON((auto.submarca = submarca.submarca))) LEFT JOIN marca ON((submarca.id_marca =
marca.id_marca)))
-> ;
Query OK, 0 rows affected (0.03 sec)
```

De esta forma, ya creada la vista, podemos ver todos los autos:

```
SELECT * FROM V1;
```

```
mysql> select * from V1;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	NULL	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	NULL	2020	BLANCO	MANUAL	50000

6 rows in set (0.00 sec)

```
mysql> SELECT * FROM V1;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	NULL	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	NULL	2020	BLANCO	MANUAL	50000

6 rows in set (0.00 sec)

Recordemos que **la vista es solo una tabla virtual que hace referencia a otra u otras tablas**. En este caso, si queremos actualizar el kilometraje de un auto, por ejemplo, del CIVIC, podemos actualizar la tabla de autos, y este cambio se verá reflejado en la vista:

```
UPDATE auto SET recorrido = 42000 WHERE vin="HDG6JBSD7FDT";
Query OK, 1 row affected (0.13 sec)
Rows matched: 1    Changed: 1    Warnings: 0
```

```
mysql> UPDATE auto SET recorrido = 42000 WHERE vin="HDG6JBSD7FDT";
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> select * from V1;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	42000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	NULL	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	NULL	2020	BLANCO	MANUAL	50000

```
6 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM V1;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	42000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	NULL	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	NULL	2020	BLANCO	MANUAL	50000

```
6 rows in set (0.00 sec)
```

Como puedes observar, el kilometraje ha cambiado, ya que se modificó la tabla base de la cual se genera la vista V1.

Modificación de la estructura de la base de datos (DROP, ALTER TABLE)

En el modelo relacional de bases de datos, se espera que la estructura de los datos no cambie con frecuencia. Sin embargo, pueden surgir situaciones en las que sea necesario modificar el esquema de la base de datos. Esto puede implicar **cambiar tablas, agregar o eliminar atributos, modificar el tipo de dato de una columna**, entre otros ajustes.

A continuación, utilizando el ejemplo de nuestra agencia de autos, exploraremos cómo realizar algunos de estos cambios.

Agregar una columna a una tabla

Supongamos que deseamos agregar una columna a la tabla `marca` para indicar el país de origen de cada marca. Por ejemplo, Volkswagen es una marca de origen alemán; Honda, Toyota y Mazda son japonesas, y Ford es estadounidense.

Para esto, y con el fin de practicar el uso de llaves foráneas, crearemos primero una tabla de países, y luego agregaremos una columna a la tabla `marca` que referencie la clave primaria de esta nueva tabla.

Primero, creamos la tabla `país`:

```
CREATE TABLE país (  
    id int NOT NULL PRIMARY KEY,  
    país VARCHAR(50)  
);
```

```
mysql> CREATE TABLE pais (  
-> id int NOT NULL PRIMARY KEY,  
-> pais VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.06 sec)
```

Después, insertamos los valores de los tres países mencionados:

```
INSERT INTO pais VALUES  
(1, "ALEMANIA"),  
(2, "JAPON"),  
(3, "ESTADOS UNIDOS");
```

```
mysql> INSERT INTO pais VALUES  
-> (1, "ALEMANIA"),  
-> (2, "JAPON"),  
-> (3, "ESTADOS UNIDOS");  
Query OK, 3 rows affected (0.02 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

Ahora sí, modificamos la tabla `marca` para agregar una columna que servirá como llave foránea y hará referencia a la tabla `país`.

Primero agregamos la columna `país`, con el tipo de dato `int`. Recuerda que el tipo de dato de la llave foránea debe coincidir con el tipo de dato de la llave primaria a la que va a referenciar:

```
ALTER TABLE marca ADD país int;
```


Luego, modificamos la tabla para agregar la **restricción** de la llave foránea que haga referencia a la tabla país:

```
ALTER TABLE marca ADD CONSTRAINT FOREIGN KEY (pais) REFERENCES pais(id);
```

```
mysql> ALTER TABLE marca ADD pais int;
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE marca ADD CONSTRAINT FOREIGN KEY (pais) REFERENCES pais(id);
Query OK, 5 rows affected (0.16 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

Como recién agregamos la columna, esta tiene el valor `NULL` en todos los registros:

```
mysql> SELECT * FROM marca;
+-----+-----+-----+
| id_marca | marca      | pais |
+-----+-----+-----+
| 1       | VOLKSWAGEN | NULL |
| 2       | HONDA      | NULL |
| 3       | TOYOTA     | NULL |
| 4       | FORD       | NULL |
| 5       | MAZDA      | NULL |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Vamos a actualizarlos:

```
UPDATE marca SET PAIS = 1 WHERE id_marca = 1;
UPDATE marca SET PAIS = 2 WHERE id_marca = 2;
UPDATE marca SET PAIS = 2 WHERE id_marca = 3;
UPDATE marca SET PAIS = 3 WHERE id_marca = 4;
UPDATE marca SET PAIS = 2 WHERE id_marca = 5;
```

Ahora ya podemos realizar una consulta para mostrar las marcas junto con su país de origen:

```
SELECT id_marca, marca, pais.pais AS pais_de_origen  
FROM marca JOIN pais ON marca.pais = pais.id;
```

```
mysql> SELECT id_marca, marca, pais.pais AS pais_de_origen  
-> FROM marca JOIN pais ON marca.pais = pais.id;
```

id_marca	marca	pais_de_origen
1	VOLKSWAGEN	ALEMANIA
2	HONDA	JAPON
3	TOYOTA	JAPON
4	FORD	ESTADOS UNIDOS
5	MAZDA	JAPON

```
5 rows in set (0.01 sec)
```

Eliminar una columna de una tabla

Supongamos que ahora deseamos eliminar la columna del país de origen de la tabla `marca` para devolver la base de datos a su estado original. Como puedes imaginar, el comando para eliminar es mucho más simple. La sintaxis es:

```
ALTER TABLE nombre_de_la_tabla DROP COLUMN nombre_de_la_columna;
```

En nuestro caso, el comando sería:

```
ALTER TABLE marca DROP COLUMN pais;
```

Sin embargo, si ejecutamos esta instrucción, obtendremos un error, ya que está trabajando como llave foránea.

```
mysql> ALTER TABLE marca DROP COLUMN pais;
ERROR 1828 (HY000): Cannot drop column 'pais': needed in a foreign key constraint 'marca_ibfk_1'
```

Como podemos ver en el resultado de la ejecución, no podemos eliminar la columna `pais` porque existe una **restricción** de llave foránea llamada “`marca_ibfk_1`”. Este es el nombre que se asignó a la restricción de llave foránea.

Por lo tanto, lo que podemos hacer es **primero eliminar la restricción de llave foránea**:

```
ALTER TABLE marca DROP FOREIGN KEY marca_ibfk_1;
```

Y después, eliminar la columna:

```
ALTER TABLE marca DROP COLUMN pais;
```

```
[mysql> ALTER TABLE marca DROP FOREIGN KEY marca_ibfk_1;
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

[mysql> ALTER TABLE marca DROP COLUMN pais;
Query OK, 0 rows affected (0.15 sec)
Records: 0 Duplicates: 0 Warnings: 0]
```

De esta forma, la tabla `marca` vuelve a su estructura original:

```
[mysql> select * from marca;
+-----+-----+
| id_marca | marca |
+-----+-----+
| 1 | VOLKSWAGEN |
| 2 | HONDA |
| 3 | TOYOTA |
| 4 | FORD |
| 5 | MAZDA |
+-----+-----+
5 rows in set (0.00 sec)
```

Cambiar el tipo de dato de un atributo de una tabla

En ocasiones, al diseñar una base de datos, es posible que no se haya elegido el tipo de dato más adecuado. Por ejemplo, supongamos que sale al mercado una nueva marca de autos cuyo nombre excede la longitud definida originalmente para el atributo `marca`.

Es muy probable que ya no nos acordemos de cómo creamos una tabla. Para revisar la estructura de una tabla, puede sernos útil el comando `DESCRIBE`:

```
DESCRIBE marca;
```

```
mysql> DESCRIBE marca;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id_marca | smallint  | NO   | PRI | NULL    |       |
| marca    | varchar(30) | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Aquí podemos darnos cuenta de que el nombre de la marca está declarado como un `varchar` de 30, es decir, una **cadena de caracteres de longitud variable** (máximo 30); sin embargo, ¿qué sucede si sale al mercado una marca cuyo nombre es de 32 caracteres? Podemos utilizar el comando `ALTER TABLE` para modificar el tipo de dato. La sintaxis es:

```
ALTER TABLE nombre_de_la_tabla MODIFY COLUMN nombre_de_
la_columna nuevo_tipo_de_dato;
```

En nuestro caso, si queremos ponerle un VARCHAR(40) a un atributo, quedaría así:

```
ALTER TABLE marca MODIFY COLUMN marca VARCHAR(40);
```

```
mysql> ALTER TABLE marca MODIFY COLUMN marca VARCHAR(40);
Query OK, 0 rows affected (0.26 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> DESCRIBE marca;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id_marca | smallint | NO   | PRI | NULL    |       |
| marca    | varchar(40) | YES |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Eliminar todos los datos de una tabla

Para concluir este tema, veamos cómo podemos eliminar una tabla completa. Aquí tenemos dos casos: uno en el que queremos borrar todos los datos de una tabla, pero conservar la estructura, y otro en el que queremos eliminar toda la tabla, es decir, tanto los datos como la estructura.

El primer caso lo vimos en la lección anterior cuando estábamos eliminando registros; mencionamos que había que tener mucho cuidado al eliminarlos y no olvidar la parte del WHERE en la instrucción DELETE FROM alguna_tabla. Por esto, si lo que queremos es deliberadamente **eliminar todos los registros de una tabla**, podemos escribir entonces la instrucción DELETE FROM alguna_tabla.

En el ejemplo de los autos, dejamos la tabla país y ya no se utilizó más. Vamos a borrar todos los registros.

La instrucción quedaría así:

```
DELETE FROM pais;
```

```
mysql> SELECT * FROM pais;
+----+-----+
| id | pais      |
+----+-----+
| 1  | ALEMANIA  |
| 2  | JAPON     |
| 3  | ESTADOS UNIDOS |
+----+-----+
3 rows in set (0.00 sec)

mysql> DELETE FROM pais;
Query OK, 3 rows affected (0.02 sec)

mysql> SELECT * FROM pais;
Empty set (0.00 sec)
```

Después de ejecutar este comando, al consultar la tabla, veremos que está vacía.

Eliminar una tabla

Si queremos eliminar completamente una tabla, tanto sus registros como su estructura, utilizamos el comando:

```
DROP TABLE nombre_de_la_tabla
```

En nuestro caso:

```
DROP TABLE pais;
```

```
[mysql> DROP TABLE pais;
Query OK, 0 rows affected (0.04 sec)

[mysql> SELECT * FROM pais;
ERROR 1146 (42S02): Table 'agencia.pais' doesn't exist
```

Al ejecutar este comando, si intentamos consultar la tabla, obtendremos un mensaje indicando que ya no existe.

Es importante tener en cuenta que si una tabla está siendo referenciada por otras (es decir, actúa como tabla padre con llaves primarias referenciadas por llaves foráneas), debemos primero eliminar esas restricciones antes de poder eliminar la tabla, como vimos anteriormente en el ejemplo del país de origen de las marcas.

Eliminar una base de datos

Finalmente, si deseamos eliminar por completo una base de datos, incluyendo todas sus tablas, vistas, y datos almacenados, usamos:

```
DROP DATABASE base_de_datos;
```

Por ejemplo, en el caso de nuestra agencia de autos, el comando sería:

```
DROP DATABASE agencia;
```


Y para terminar...

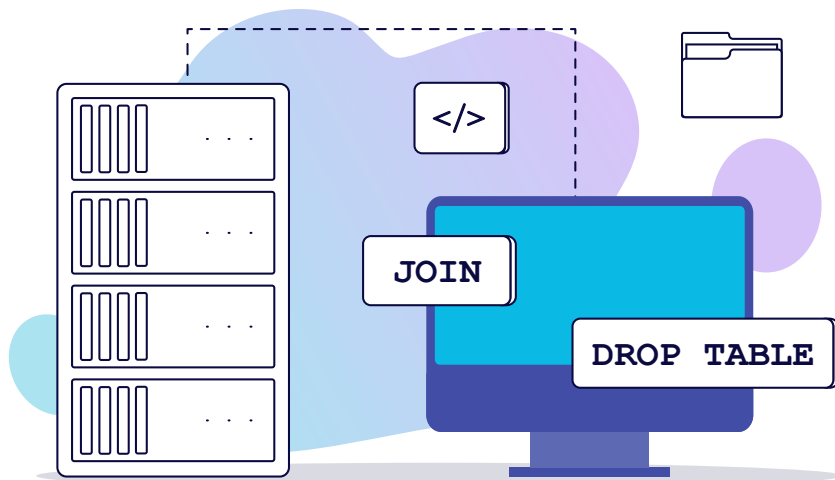
A lo largo de este tema, exploramos herramientas clave para trabajar con bases de datos de manera eficiente. Las consultas anidadas nos permiten descomponer problemas complejos en consultas más manejables, mientras que las vistas facilitan el acceso y la reutilización de información sin comprometer la integridad de los datos.

Además, revisamos cómo modificar la estructura de la base de datos con comandos como `DROP TABLE` y `ALTER TABLE`, lo que nos brinda flexibilidad en la administración de datos en entornos dinámicos.

Para reforzar estos conocimientos, reflexiona sobre las siguientes preguntas:

- ¿Cuándo sería preferible usar una consulta anidada en lugar de una `JOIN`?
- ¿En qué escenarios una vista podría ser más eficiente que una tabla real?
- ¿Qué precauciones debemos tomar antes de eliminar una columna o una tabla con `DROP TABLE` o `ALTER TABLE`?

Continúa con el siguiente recurso para poner a prueba los conocimientos adquiridos en esta lección.



| Glosario

- **ALTER TABLE:** Comando SQL que permite modificar la estructura de una tabla existente, ya sea para agregar o eliminar columnas, cambiar el tipo de datos de un atributo o añadir restricciones como llaves foráneas.
- **DELETE FROM:** Instrucción utilizada para eliminar registros de una tabla. Si no se especifica una cláusula `WHERE`, todos los registros de la tabla serán eliminados, pero la estructura de la tabla permanecerá intacta.
- **DESCRIBE:** Comando utilizado para visualizar la estructura de una tabla, mostrando los nombres de columnas, tipos de datos, restricciones, entre otros detalles útiles para el mantenimiento de la base de datos.
- **DROP DATABASE:** Comando que elimina completamente una base de datos, incluyendo todas sus tablas, vistas, datos y cualquier otro objeto asociado. Debe usarse con extrema precaución.
- **DROP TABLE:** Instrucción que elimina por completo una tabla de la base de datos, incluyendo todos sus registros y estructura. Es una acción irreversible que requiere precaución.
- **Llave foránea (*foreign key*):** Restricción que establece una relación entre dos tablas, permitiendo que una columna (o conjunto de columnas) haga referencia a la llave primaria de otra tabla, asegurando la integridad referencial.
- **Llave primaria (*primary key*):** Atributo o conjunto de atributos que identifica de manera única cada fila de una tabla. No puede tener valores nulos y debe ser único.
- **UPDATE:** Comando SQL que permite modificar valores existentes en los registros de una tabla, utilizando generalmente una cláusula `WHERE` para especificar qué filas serán afectadas.

Material adicional

- MySQL. (2024). *Using Views* [Uso de vistas]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 15 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/views.html>

Nota: Los enlaces a las páginas web incluidos en esta sección fueron consultados y verificados por última vez el 15 de abril de 2025 y su disponibilidad depende del autor.

Bibliografía

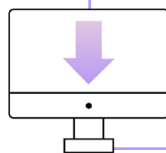
Elmasri, R. & Navathe, S. B. (2016). *Fundamentals of database systems* [Fundamentos de los sistemas de bases de datos] (7th ed.). Pearson Education.

MySQL. (2024). *Working with NULL values* [Trabajando con valores NULL]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 15 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/working-with-null.html>

MySQL. (2024). *Subqueries* [Subconsultas]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 15 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/subqueries.html>

MySQL. (2024). *ALTER TABLE Statement* [Declaración ALTER TABLE]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 15 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/alter-table.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 15 de abril de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.